GNU Binutils

A Collection of Binary Tools

With years of experience as a systems programming engineer, I've realised that to be a great artist of programming, you must understand the basics of source code, output files, linkers etc. So let's dive deep into linkers, ELF and assembly code. We have a rich set of tools to create, examine, link and dissect binary files, called 'binutils'.

Let's start with the *'ar'* utility, which is used to create, extract and modify archives. Its main use is in building static libraries, which we will examine with an example. We will create two *.c* files, with a function defined in each. The first is *foo.c :*

```
void foo(int *i){
     *i=*i+5;
}
```

The second is *bar.c:*

```
void bar(int *i){
     *i=*i*5;
}
```

Now, let's compile these two files, using the *'-c'* GCC option to only create object files and not link them. After compilation, we have the object files, as shown:

```
$ gcc -c foo.c && gcc -c bar.c
$ ls *.o
bar.o  foo.o
```

Most of the time, we do not write code for a program from scratch, but use already available source or compiled code. For example, in C, we use the *'printf()'* function, from a *library* of pre-compiled functions. Libraries are of two types, static and dynamic. Dynamic libraries need to be linked to at run-time; if one is missing, the application won't run. In cases where we want a program to be 'independent' and not require any other libraries to be installed on the system before it can run, we need to resolve external functions and variables at compile time, and copy them into the program binary. This removes the run-time dependency on the library. For this purpose, we create static libraries—archive files of one or more object files. The *'ar'* tool can archive binary files, and is used to create such static libraries—actually, to create, modify and extract archive files. You may ask, what is the difference from the tar (tape archive) tool, which also can archive binary and other types of files, and that too, with compression? The answer is, *'ar'* creates a symbol table inside the output file, whereas *tar* doesn't; *'ar'* yields a collection of symbols, and *tar* a collection of files. You can get more details from the *ar* man page.

So let's create our static library. Its name should be in the format *'liblibraryname.a'*. The *'lib'* prefix is necessary, and the extension must be *'.a'* as per UNIX standards (*.so* is used for shared libraries):

```
$ ar -cvq libfoobar.a foo.o bar.o
a - foo.o
a - bar.o
$ file libfoobar.a
libfoobar.a: current ar archive
```